How-To Library

Informatica

Using Workflow Parameters and Variables to Write to a Dynamic Target File

© Copyright Informatica LLC 2013, 2021. Informatica Corporation. No part of this document may be reproduced or transmitted in any form, by any means (electronic, photocopying, recording or otherwise) without prior consent of Informatica Corporation. All other company and product names may be trade names or trademarks of their respective owners and/or copyrighted materials of such owners.

Abstract

You can use parameters and variables in a workflow designed in the Developer tool to make the workflow more flexible. This article describes how to use workflow parameters and variables to dynamically determine the name of the target file at run time.

Supported Versions

- Data Quality 9.5.0
- PowerCenter Big Data Edition 9.5.1

Table of Contents

Overview. . . . 2
Scenario. . . . 3
Step 1. Create a Flat File Data Object Parameter for the Target File Name. . . . 3
Step 2. Create the Workflow and Add the Mapping Task. . . . 6
Step 3. Create a Workflow Parameter for the Department Name. . . . 6
Step 4. Create a Workflow Variable for the Dynamic Target File Name. . . . 7
Step 5. Add an Assignment Task to Assign a Value to the Variable. . . . 8
Step 6. Override the Flat File Data Object Parameter. . . . 10
Step 7. Deploy the Workflow. . . . 10
Step 8. Create the Parameter File. . . . 11
Step 9. Run the Workflow with the Parameter File. . . . 11

Overview

You can use parameters and variables in a workflow designed in the Developer tool to make the workflow more flexible.

A workflow parameter is a constant value that you define before the workflow runs. Parameters retain the same value throughout the workflow run. You define the value of the parameter in a parameter file. The Data Integration Service applies parameter values when you run a workflow from the command line and specify a parameter file. When you create parameters in a workflow, you can run a workflow again with different parameter values. This can reduce the overhead of creating multiple workflows when only certain attributes of a workflow need to be changed.

For example, you create a workflow that includes a Notification task. You want the task to send an email to yourself as you test the workflow but send an email to an administrator when you run the workflow in production. You create a workflow parameter that defines the email address to which you want to send an email. Use the workflow parameter in the recipients field for the Notification task. Create one parameter file that sets the email address to your address. Create another parameter file that sets the email address to the administrator's address. When you run the workflow from the command line, use the appropriate parameter file for the workflow run.

A workflow variable represents a value that can change during a workflow run. Use workflow variables to reference values and record run-time information. A user-defined workflow variable captures task output or captures criteria that you specify. When you create a user-defined workflow variable, you enter an initial value. The Data Integration Service uses the initial value of the variable when the workflow starts. You add an Assignment task to the workflow to assign another value to the variable. The Data Integration Service uses the assigned value for the variable during the remainder of the workflow.

For example, you create a counter variable and set the initial value to zero (0). In the Assignment task, you increment the variable by setting the variable to its current value plus one (1).

Scenario

You have a single mapping that multiple departments need to run. For each mapping run, you need the mapping to write to a flat file named with the department name and a time stamp with the current date. You do not want to create a unique mapping for each department because of the overhead of maintaining multiple mappings.

Solution

Create a workflow to run the mapping. Design the workflow to use parameters and variables to dynamically determine the file name at run time.

Design the flat file data object to use a parameter for the output file name. Create a workflow and add the mapping to the workflow as a Mapping task. Create a workflow parameter to represent the department name. When a user runs the workflow from the command line, the user specifies a parameter file that includes the department name. Create a workflow variable to represent the dynamic target file name. Use an Assignment task to assign the value of the department name parameter and the current date to the workflow variable. In the Mapping task, override the flat file data object parameter with the workflow variable that captures the department name and current date.

Deploy the workflow, and then use the infacmd wfs ListWorkflowParams command to create a parameter file that lists the default value of the department name workflow parameter. Edit the parameter file to replace the default parameter value with the appropriate department name. Then, use the infacmd wfs StartWorkflow command to run the workflow with the edited parameter file.

Step 1. Create a Flat File Data Object Parameter for the Target File Name

A mapping parameter represents a constant value that can change between mapping runs, such as connections, source file directories, or target file names. Create a user-defined parameter in the flat file data object to represent the flat file name that the data object writes to. Apply the parameter to the output file name property in the flat file data object.

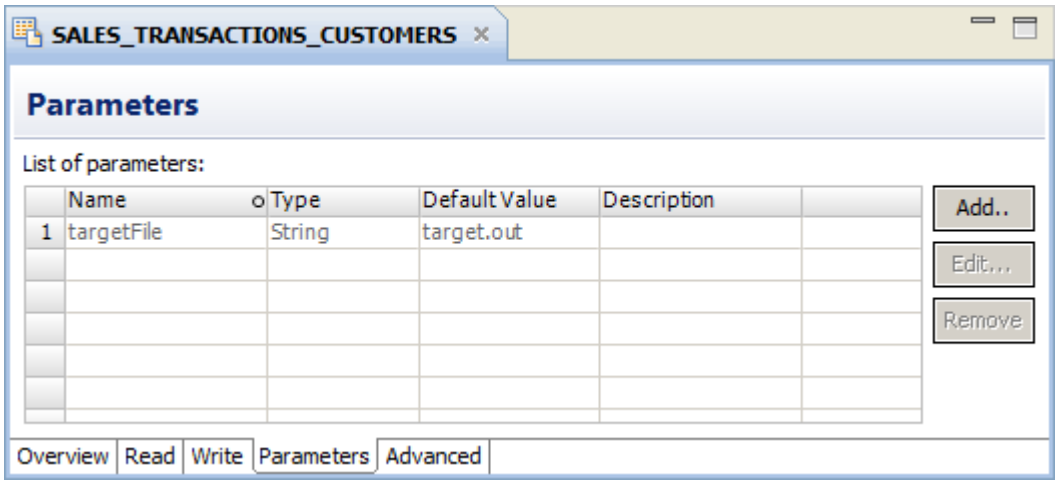
1. Open the flat file data object used as the target in the mapping.
2. Click the **Parameters** view.
3. Click **Add**.

   The **Add Parameter** dialog box appears.
4. Enter the parameter name.

   For example, enter targetFile as the parameter name.
5. Select **String** for the parameter type.
6. Enter a default value for the parameter.

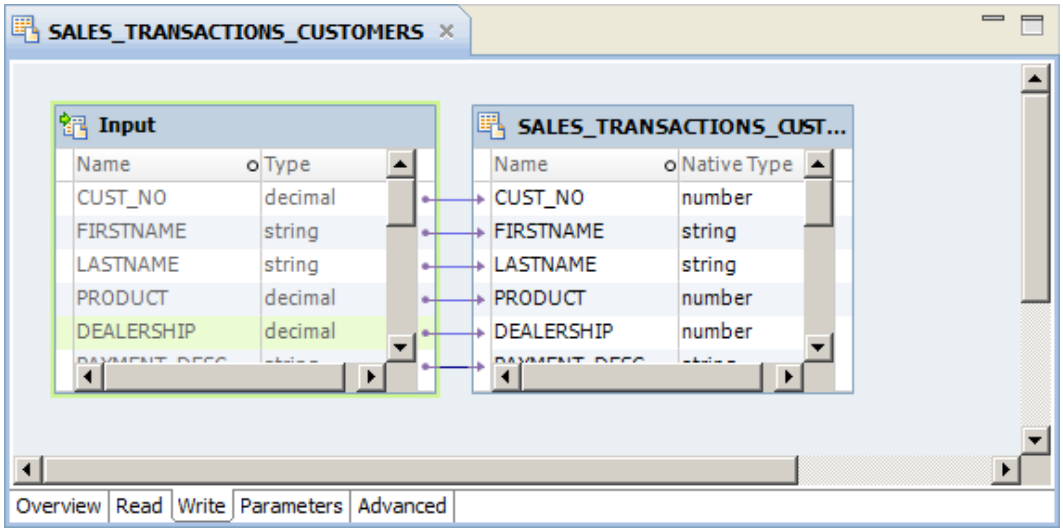
   For example, enter target.out as the default value.
7. Click **OK**.

The Developer tool adds the parameter to the list of parameters for the flat file data object.



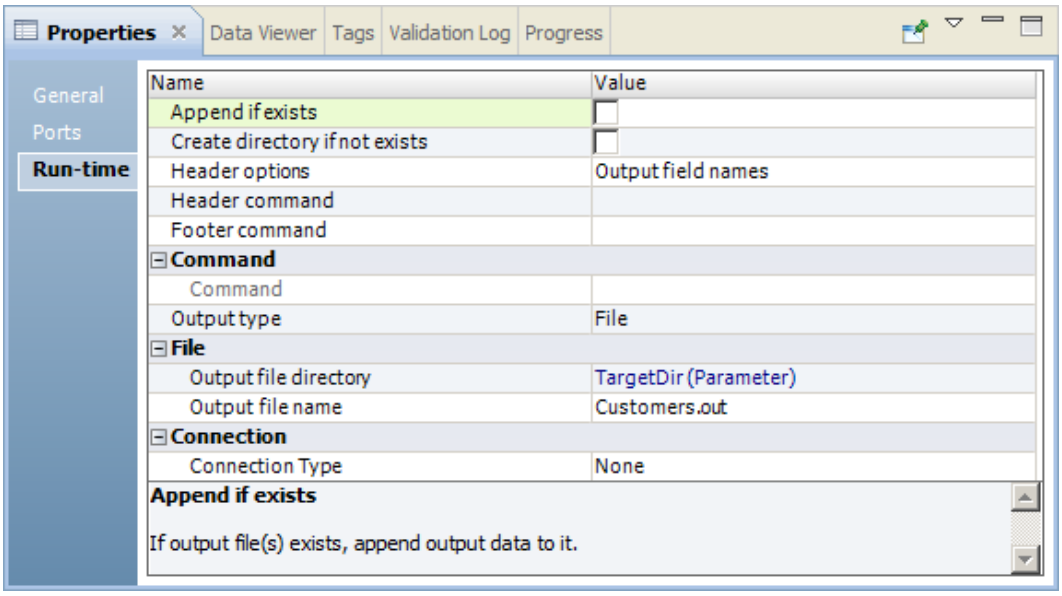
8. Click the **Write** view.
9. Select the Input transformation to view run-time properties that the Data Integration Service uses when it writes data to the file.

   The following figure shows the Input transformation selected in the **Write** view:



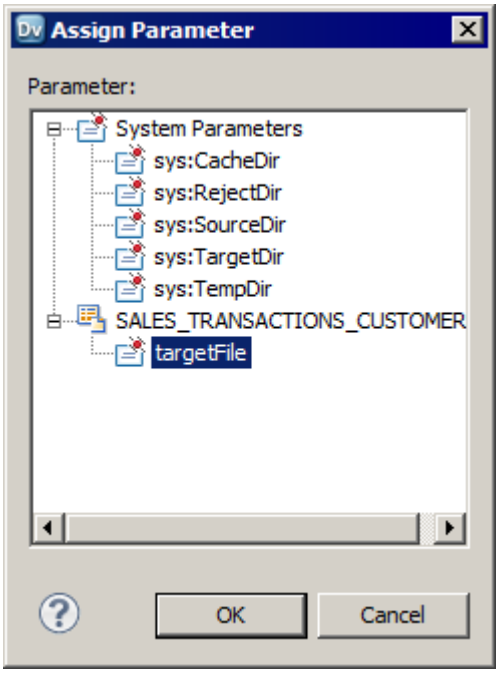
10. In the **Properties** view, click the **Run-time** tab.

The following figure shows the **Run-time** tab:



11. In the **Value** column for the **Output file name** property, click **Assign Parameter**.

    The **Assign Parameter** dialog box appears.



12. Select the parameter created for the flat file data object.

    For example, select targetFile.

13. Click **OK**.

    The Developer tool displays the parameter name as the value of the **Output file name** property.

14. Click **File** > **Save**.

Step 2. Create the Workflow and Add the Mapping Task

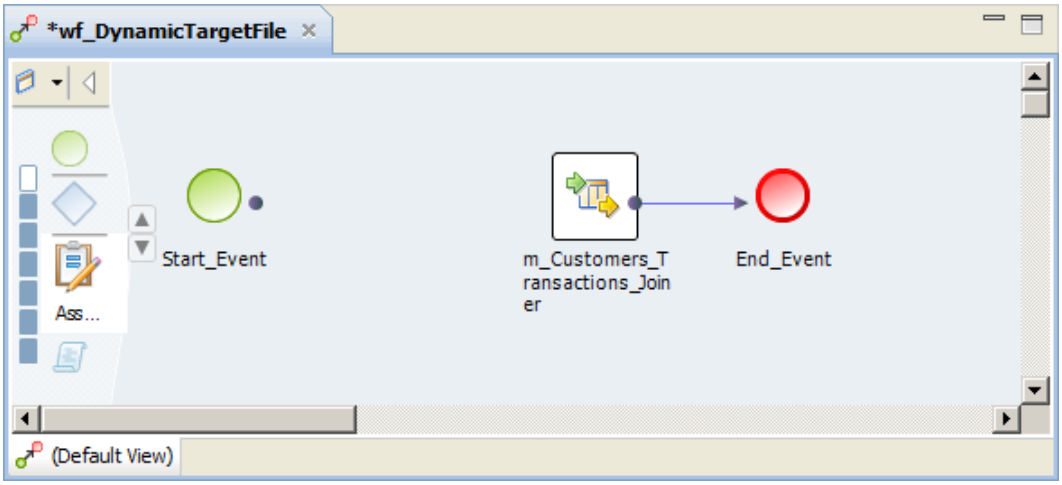
Create the workflow, and then add a Mapping task that runs the mapping from the workflow.

1. In the **Object Explorer** view, select the folder where you want to create the workflow.
2. Right-click and select **New** > **Workflow**.
3. In the **Workflow** dialog box, enter wf_DynamicTargetFile for the workflow name and click **Finish**.

   A workflow with a Start event and an End event opens in the editor.
4. In the **Object Explorer** view, select the mapping that writes to the flat file data object and drag it to the editor.

   A Mapping task with the same name as the mapping appears in the editor.
5. Add a sequence flow to connect the Mapping task to the End event.

   The following figure shows the Mapping task connected to the End event:



Step 3. Create a Workflow Parameter for the Department Name

Create a workflow parameter to represent the department name. After you add an Assignment task to the workflow, you use the parameter in an Assignment task expression to add the current date to the department name value.

1. In the **Properties** view of the workflow, click the **Parameters** tab.
2. Click **Add**.

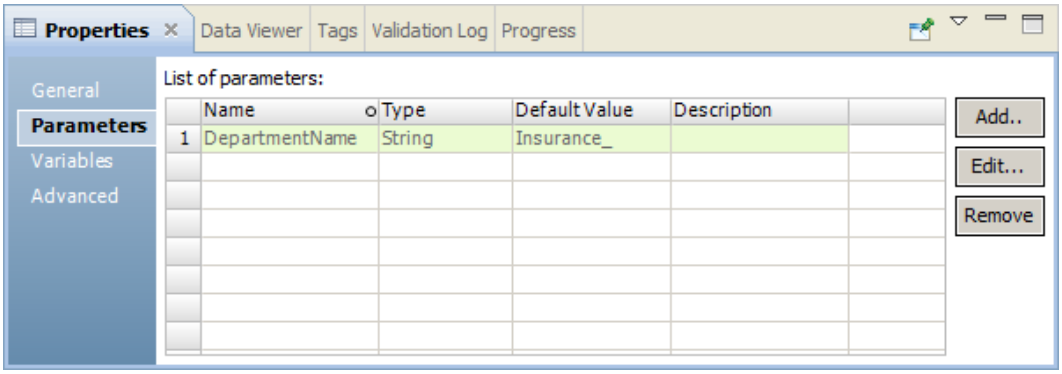
   The **Add Parameter** dialog box appears.
3. Enter the parameter name.

   For example, enter DepartmentName as the parameter name.
4. Select **String** for the parameter type.
5. Enter a default value for the parameter.

   For example, enter Insurance_ as the default value.

6. Click **OK**.

   The Developer tool adds the parameter to the list of parameters for the workflow.



Step 4. Create a Workflow Variable for the Dynamic Target File Name

Create a workflow variable to represent the dynamic target file name. After you add an Assignment task to the workflow, you assign the value of the department name parameter and the current date to the variable.

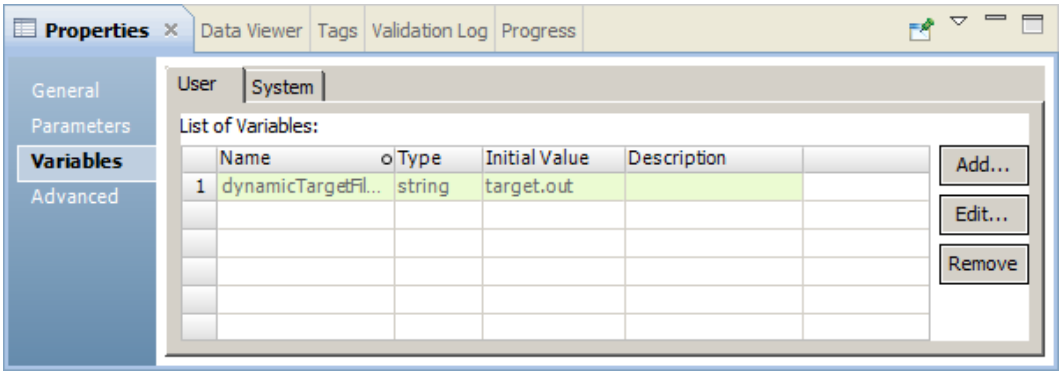
1. In the **Properties** view of the workflow, click the **Variables** tab.
2. Click **Add**.

   The **Add Variable** dialog box appears.
3. Enter the variable name.

   For example, enter dynamicTargetFileName as the variable name.
4. Select **String** for the variable type.
5. Enter an initial value for the variable.

   For example, enter target.out as the initial value.
6. Click **OK**.

   The Developer tool adds the variable to the list of variables for the workflow.



Step 5. Add an Assignment Task to Assign a Value to the Variable

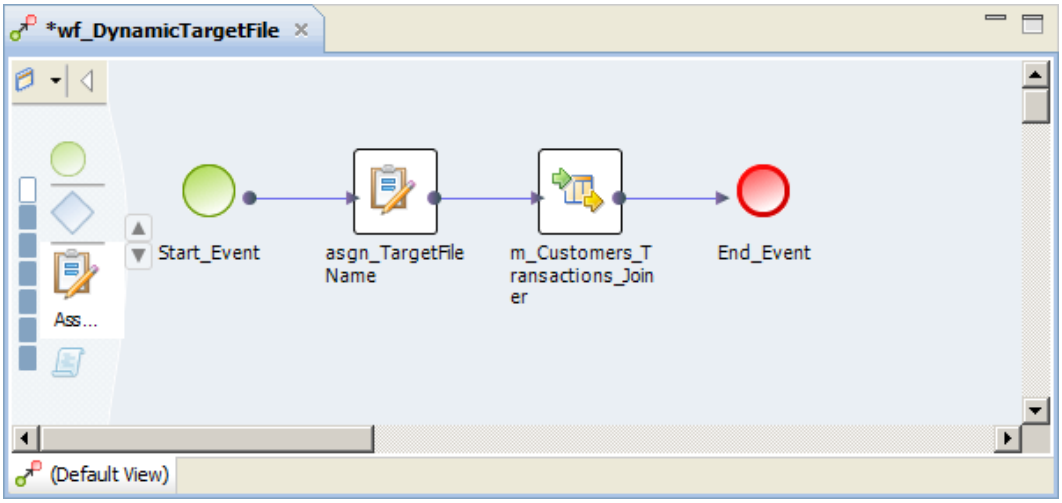
An Assignment task assigns a value to a user-defined workflow variable. Add an Assignment task before the Mapping task in the workflow. Use an expression in the Assignment task to assign the value of the department name parameter and the current date to the workflow variable.

1. Right-click the workflow editor and select **Add Workflow Object**.
2. Select **Assignment Task** and click **OK**.

   The Assignment task appears in the editor.
3. In the **Properties** view for the Assignment task, click the **General** tab.
4. Rename the Assignment task.

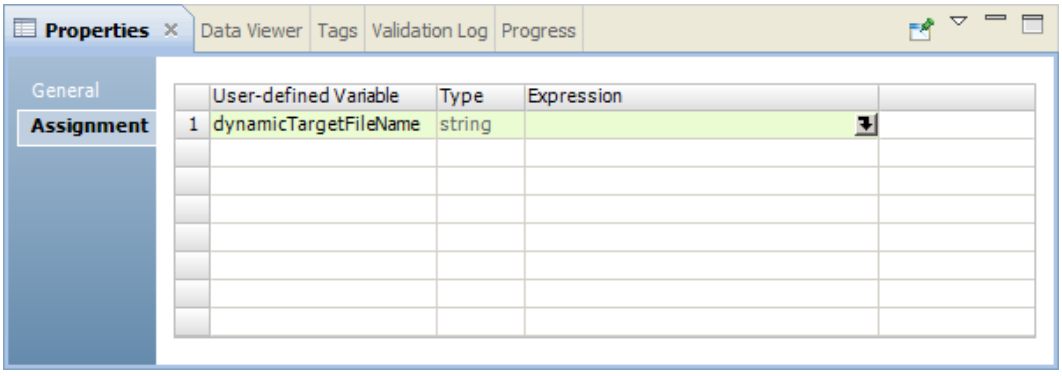
   For example, name the Assignment task asgn_TargetFileName.
5. Select the Assignment task and move the task between the Start event and the Mapping task.
6. Add a sequence flow to connect the Start event to the Assignment task.
7. Add another sequence flow to connect the Assignment task to the Mapping task.

   The following figure shows the connected events and tasks in the workflow:



8. In the **Properties** view for the Assignment task, click the **Assignment** tab.
9. In the **User-defined Variable** column, select the variable that you created for the target file name.

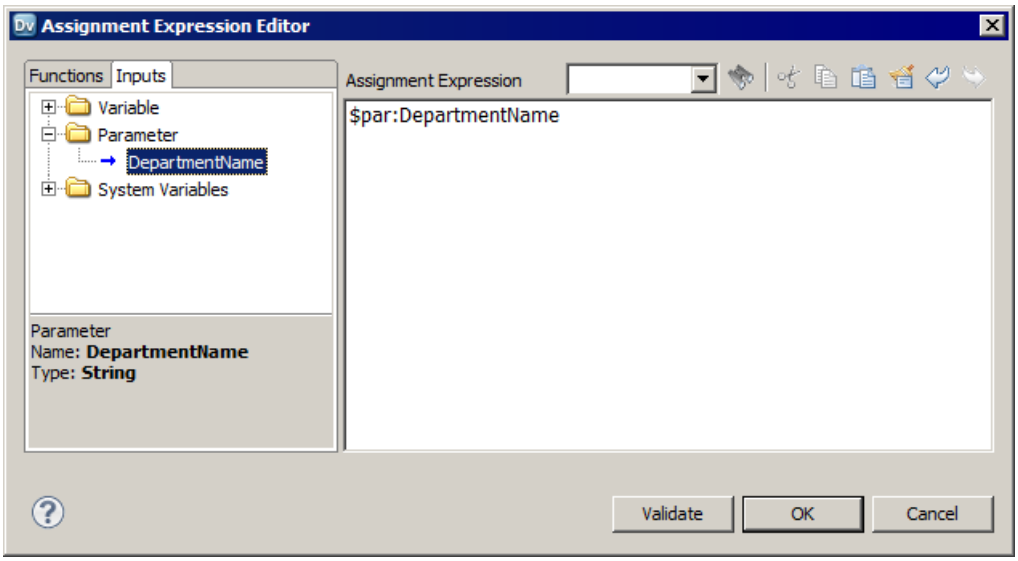
   The following figure shows the selected dynamicTargetFileName variable:



10. In the Expression column, click the **Open** button (↧) to open the **Assignment Expression Editor**.

11. Use the point-and-click method in the **Assignment Expression Editor** to enter the expression.
    a. In the **Inputs** tab, expand the **Parameter** folder.
    b. Double-click the parameter that you created for the department name.

       The Developer tool adds the workflow parameter using the required syntax for expressions.

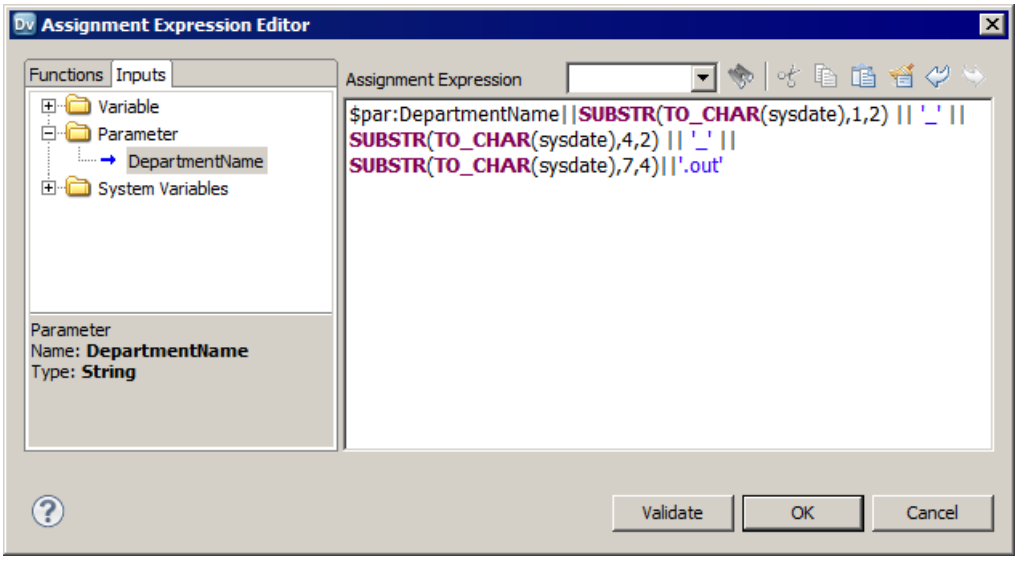


    c. Use the concatenation operator (||), the SUBSTR function, the TO_CHAR function, and the built-in variable sysdate to add the current date to the workflow parameter value.

       For example, the following expression adds the current month, day, and year to the workflow parameter value:

```
$par:DepartmentName||SUBSTR(TO_CHAR(sysdate),1,2) || '_' || SUBSTR(TO_CHAR(sysdate),
4,2) || '_' || SUBSTR(TO_CHAR(sysdate),7,4)||'.out'
```

       The following figure shows the completed expression in the Assignment Expression Editor:



    d. Click **Validate** to validate the expression.
    e. Click **OK** to close the **Expression Editor**.

The Developer tool displays the workflow variable and the expression assigned to the variable.

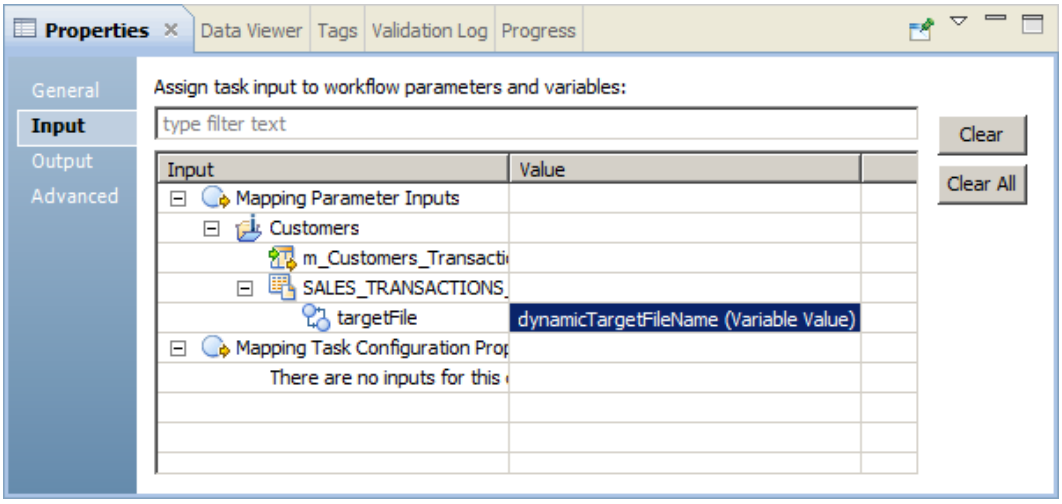
Step 6. Override the Flat File Data Object Parameter

You can override user-defined mapping parameter values during a workflow run by assigning mapping parameters to workflow parameters or variables in the Mapping task Input tab. You override mapping parameters with workflow variables when you want to use workflow run-time data for the mapping parameter value.

In the Mapping task, override the flat file data object parameter with the workflow variable that captures the department name and current date. When the Mapping task runs the mapping, the mapping uses the value dynamically assigned to the workflow variable for the output file name.

1. Select the Mapping task in the editor.
2. In the **Properties** view, click the **Input** tab.
3. Under **Mapping Parameter Inputs**, find the parameter created for the flat file data object to represent the target file name.
4. In the **Value** column, select the arrow, and then double-click the user-defined workflow variable that represents the dynamic target file name.

   The following figure shows that the flat file data object parameter named targetFile is assigned to the workflow variable named dynamicTargetFileName:



5. To verify that the workflow is valid, right-click the editor and select **Validate**.
6. Click **File** > **Save** to save the workflow.

Step 7. Deploy the Workflow

To run an instance of the workflow, you add the workflow definition to an application. Then, you deploy the application to the Data Integration Service.

You deploy workflows to allow users to run workflows with a parameter file using the infacmd wfs startWorkflow command. Use the Developer tool to deploy workflows. You deploy workflows using the same procedure that you use to deploy other Model repository objects.

Step 8. Create the Parameter File

The infacmd wfs ListWorkflowParams command lists the parameters used in a deployed workflow and the default value for each parameter. Use the output of this command to create a parameter file.

The command lists all parameters in a project top-level element. You can edit the parameter default values in the project element to define the values for a workflow in the project that is deployed to any application. Or, you can copy the project element into an application element to define the values for a specific workflow in a specific deployed application.

1. Run the infacmd wfs ListWorkflowParams command to list all parameters used in a deployed workflow and the default value for each parameter.

   For example, the following command lists the parameters in the workflow wf_DynamicTargetFile in the file "MyOutputFile.xml":

   ```
   infacmd wfs ListWorkflowParams -dn MyDomain -sn MyDataIntSvs -un MyUser -pd MyPassword -a
   MyApplication -wf wf_DynamicTargetFile -o MyOutputFile.xml
   ```

   The Data Integration Service lists all parameters in the workflow with their default values in a project top-level element. For example, the following sample parameter file lists the default value for the DepartmentName parameter:

   ```
   <?xml version="1.0" encoding="UTF-8"?><root xmlns="http://www.informatica.com/
   Parameterization/1.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema">
       <!--
       <application name="MyApplication">
           <workflow name="wf_DynamicTargetFile">
               <!- -Specify deployed application specific parameters here.- ->
           </workflow>
       </application>
       -->
       <project name="MyProject">
           <workflow name="wf_DynamicTargetFile">
               <parameter name="DepartmentName">Insurance_</parameter>
           </workflow>
       </project>
   </root>
   ```

2. Edit the XML file and replace the parameter default value with the department name that you want to use when you run the workflow.

   If you want to define the values for the workflow in a specific application, copy the project top-level element into an application top-level element.

3. Save the XML file.

Step 9. Run the Workflow with the Parameter File

Use the infacmd wfs StartWorkflow command to run the workflow with the parameter file.

For example, the following command runs the workflow wf_DynamicTargetFile using the parameter file "MyParamFile.xml":

```
infacmd wfs StartWorkflow -dn MyDomain -sn MyDataIntSvs -un MyUser -pd MyPassword -a
MyApplication
-wf wf_DynamicTargetFile -pf MyParamFile.xml
```

The Data Integration Service reads the parameter file at the start of the workflow run, and resolves the DepartmentName parameter to the value set in the parameter file.

Author

Alison Taylor

Acknowledgements

The author would like to thank the Informatica Platform development and QA teams for their help in completing this article.